

Predicting UEFA Champions League Knockout Winners Using Machine Learning

Team =TVD [Janhavi Wagh]

- **Competition:** AI Club DC Hackathon 2024–25
- **Task:** Kaggle-style prediction challenge
- **Models Used:** Logistic Regression, Random Forest, Gradient Boosting

1. Problem Statement

The objective of this project is to predict the winners of UEFA Champions League knockout matches for future seasons using historical match data.

2. Dataset Description

The dataset is provided in JSON format and consists of three main components:

- **Training Data (train.json):** Historical Champions League match data up to the 2017 season.
- **Test Data (test_matchups.json):** Future knockout-stage matchups without ground-truth labels.
- **Sample Submission (sample_submission.csv):** Defines the required output structure for Kaggle submission.

3. Feature Engineering

Feature engineering was carried out in two structured stages to capture both domestic and continental performance of teams.

3.1 Domestic League Features

In the first stage, features were derived from teams' **domestic league performances**. These features capture a team's baseline strength and consistency over regular-season play.

These domestic features provide a stable estimate of a team's overall quality, as domestic leagues offer a larger sample of matches compared to knockout tournaments.

3.2 Champions League (UCL) Pedigree Features

In the second stage, **Champions League-specific features** were incorporated to capture performance under high-pressure continental competition.

Certain high-level attributes such as **pedigree, tournament experience, and stage importance** were manually encoded to reflect qualitative differences between teams that are difficult to capture purely from match statistics.

3.3 Match-Level Feature Construction

For each matchup, feature differences between the competing teams were computed using both domestic and UCL-specific features. This ensured that predictions account for both baseline team strength and performance in elite European competition. All engineered features were saved and reused during inference to maintain consistency and prevent data leakage.

4. Exploratory Data Analysis (EDA)

Before model training, exploratory data analysis was performed to understand the structure and behavior of the dataset. This included:

- Examining class distribution (win vs loss outcomes)
- Analyzing feature ranges and summary statistics
- Checking correlations between engineered features
- Identifying potential data imbalance and variance across seasons

EDA helped validate the relevance of engineered features and informed modeling decisions such as model choice, probability calibration, and the need for robust validation strategies.

5. Modeling and Ensembling Approach

The modeling strategy was designed with a balance between interpretability, robustness, and ranking performance, which is particularly important for competitions evaluated using ROC-AUC.

5.1 Baseline and Individual Models

Multiple supervised learning models were explored:

- **Logistic Regression** was used as a strong baseline due to its interpretability and stable probability estimates.
- **Random Forest** was employed to capture non-linear interactions among features and reduce variance.
- **Gradient Boosting** was used to model complex relationships through sequential error correction.

Each model was trained on the same feature set and evaluated on a held-out validation split to ensure fair comparison.

5.2 Ensembling Strategy

Rather than relying on a single model, an ensembling perspective was adopted. Predictions from multiple models were combined using **weighted soft voting**, where each model contributed probabilistic outputs with empirically chosen weights. This approach improved robustness and reduced sensitivity to individual model variance.

The ensemble demonstrated improved ranking stability compared to individual models, making it well-suited for ROC–AUC–based evaluation.

5.3 Evaluation Metrics: Accuracy, ROC Curve, and AUC

Model performance on the validation set was assessed using multiple metrics:

- **Accuracy** was used as a basic sanity check to ensure reasonable classification behavior.
- **(ROC) curve** was used to analyze the trade-off between true positive rate and false positive rate across probability thresholds.
- (ROC–AUC) was used as the primary evaluation metric, as it measures the model's ability to correctly rank winning teams irrespective of a fixed decision threshold.

5.4 Probability Calibration and Final Model Selection

We explored probability calibration techniques such as **Platt scaling** to improve the interpretability of predicted probabilities. However, calibration requires fitting an additional model on labeled validation data, which can introduce validation leakage if used in the final submission. Moreover, calibration primarily improves probability correctness rather than ranking quality and was observed to slightly reduce **ROC–AUC**, the competition's evaluation metric.

Therefore, to ensure fairness, avoid leakage, and preserve ranking performance, the **raw ensemble probabilities obtained via weighted soft voting of multiple models** were used for the final predictions.

Overall, the modeling approach emphasizes controlled complexity, reproducibility, and strong ranking performance over aggressive optimization.

5. Training and Validation Strategy

Models were trained exclusively on historical data up to the 2017 season. Validation was performed using season-based splits to reflect real-world prediction scenarios and to avoid any leakage of future information into training.

Standard scaling was applied to numerical features, and the fitted scaler was saved and reused during the inference stage.

6. Inference on Test Matchups

The trained and calibrated model was applied to the provided `test_matchups.json` file to generate predictions. Using the same feature engineering and preprocessing pipeline as training, the model produced:

- Predicted winner for each matchup
- Corresponding win probability

These predictions were then used to simulate the entire knockout bracket, where winners of earlier rounds advanced to subsequent rounds.

It is important to note that this step involves **prediction (inference)** and not model evaluation, as ground-truth labels are not available for the test data.

7. Submission Preparation

The final predictions were inserted into the provided `sample_submission.csv` template. Care was taken to:

- Preserve the exact column names and structure
- Maintain correct row ordering
- Ensure valid JSON formatting for nested knockout rounds

This ensured full compatibility with Kaggle's submission requirements.

8. Results and Observations

The model generated consistent win probabilities across knockout rounds. While ensemble models showed slightly higher variance in predictions, Logistic Regression demonstrated stable behavior, making it suitable for bracket-style simulations.

initially when I went only one model didn't clean the properly there were lot of errors and Accuracy was low but with time when I added multiple parameters used write tools , accuracy went significantly up .

Experiments :

Initially started the model with basic models such as Logistic regression and didn't clean the properly also selection of parameters wasn't well enough and millions of error was there in code , It took a lot time understand the data and actually what pipeline to follow.